Dimensione dell'input

Indice

Dimensione dell'input

Notazione asintotica: un richiamo

Istanza di input costituita da un numero naturale

Istanza di input costituita da due numeri naturali

Istanza di input costituita da un array di numeri naturali
- Un limite superiore
- Un limite inferiore
- Componenti di dimensione costante
- Due varianti dell'input array

Dimensione dell'input e numero di passi

Perché serve una misura dell'input?

- Per confrontare due algoritmi che risolvono lo stesso problema **contiamo quante operazioni** eseguono
- Questo numero cresce con l'input:
  - ordinare un array di un milione di componenti richiede più tempo che ordinare un array di dieci componenti
  - verificare se un numero di venti cifre è primo richiede più tempo che verificarlo per un numero di due cifre
- Per rendere preciso questo discorso serve una misura di quanto sia *grande* un input

Il piano del capitolo

- La **dimensione dell'input** è il numero di bit necessari per scrivere l'input in binario, come nel capitolo «Rappresentazione dei numeri»
- Definiamo la dimensione dell'input e richiamiamo la notazione asintotica del capitolo «Notazione asintotica»
- Calcoliamo la dimensione dell'input, e il suo ordine di crescita, per un numero naturale, due numeri naturali e un array di numeri naturali
- Mostriamo perché la distinzione è importante: lo stesso numero di passi può essere piccolo o enorme rispetto alla dimensione dell'input, a seconda che n sia il *valore* di un numero o il *numero di componenti* di un array

Dimensione dell'input

Introduzione: Dimensione dell'input

- Definiamo la **dimensione dell'input** di un algoritmo
- È la misura con cui esprimeremo il tempo di esecuzione di ogni algoritmo del corso
- Ricordiamo quanti bit servono a un numero naturale, e calcoliamo la dimensione dell'input di tre piccole istanze

Definizione 1: dimensione dell'input

*La **dimensione dell'input** $|I|$ di un algoritmo è il **numero totale di bit necessari per rappresentare in binario una data istanza di input** I.*

La dimensione dell'input come funzione

- La dimensione dell'input è una funzione che associa a ogni istanza di input un numero naturale, pari al numero totale di bit necessari per rappresentare in binario l'istanza di input

- Consideriamo algoritmi le cui istanze di input sono costituite
  - da uno o più numeri naturali, oppure
  - da array di numeri naturali

Quanti bit servono a un numero naturale

- Nel capitolo «Rappresentazione dei numeri» abbiamo visto che il numero minimo di cifre necessarie per scrivere un numero naturale $n \geq 1$ in base 2 è

$$\lfloor \log_2 n \rfloor + 1$$

- Equivalentemente, un numero naturale n si scrive con esattamente k bit se e solo se

$$2^{k-1} \leq n \leq 2^k - 1.$$

Esempio (1/3)

Un numero naturale, $n = 37$.

- Poiché $2^5 = 32 \leq 37 < 64 = 2^6$, si ha $\lfloor \log_2 37 \rfloor = 5$
- Quindi

$$37 = (100101)_2, \qquad |I| = \lfloor \log_2 37 \rfloor + 1 = 6.$$

Esempio (2/3)

Due numeri naturali, $n = 37$ e $m = 200$.

- Poiché $2^7 = 128 \leq 200 < 256 = 2^8$, il numero $200 = (11001000)_2$ richiede 8 bit
- Il numero 37 richiede 6 bit, e

$$|I| = 6 + 8 = 14.$$

Esempio (3/3)

L'array $a = [6, 1, 3, 8]$.

- Le sue componenti sono $6 = (110)_2$, $1 = (1)_2$, $3 = (11)_2$ e $8 = (1000)_2$
- Quindi

$$|I| = 3 + 1 + 2 + 4 = 10.$$

Dai valori esatti alla crescita

La dimensione esatta dell'input dipende
dai valori specifici dell'istanza.

Nell'analisi degli algoritmi ci interessa come essa *cresce*,
e per questo usiamo la **notazione asintotica**.

Riepilogo

- La dimensione dell'input $|I|$ è il **numero totale di bit** necessari per scrivere in binario l'istanza di input
- Un numero naturale $n \geq 1$ richiede $\lfloor \log_2 n \rfloor + 1$ bit, cioè esattamente k bit se e solo se $2^{k-1} \leq n \leq 2^k - 1$
- La dimensione dell'input di un'istanza è la somma dei bit di tutti i suoi numeri: 6 per 37, 14 per 37 e 200, 10 per $[6, 1, 3, 8]$

Notazione asintotica: un richiamo

Introduzione: Notazione asintotica: un richiamo

- La notazione asintotica è definita e studiata nel capitolo «Notazione asintotica»
- Qui richiamiamo soltanto le definizioni e i due fatti che si usano in questo capitolo
- Sono gli strumenti con cui descriviamo come cresce la dimensione dell'input

Il contesto

- Come nel capitolo «Notazione asintotica», f e g sono funzioni
  - definite sui numeri naturali $n \geq 1$
  - a valori reali non negativi

- La notazione confronta i loro **ordini di crescita** per tutti gli n sufficientemente grandi

La notazione asintotica (1/2)

- $f(n) = O\big(g(n)\big)$ (**limite superiore**) se esistono $c \in \mathbb{R}_{>0}$ e $n_1 \in \mathbb{N}$ tali che

$$0 \leq f(n) \leq c\, g(n), \qquad \forall n \geq n_1;$$

- $f(n) = \Omega\big(g(n)\big)$ (**limite inferiore**) se esistono $c \in \mathbb{R}_{>0}$ e $n_1 \in \mathbb{N}$ tali che

$$0 \leq c\, g(n) \leq f(n), \qquad \forall n \geq n_1;$$

La notazione asintotica (2/2)

- $f(n) = \Theta\big(g(n)\big)$ (**entrambi**) se esistono $c_1, c_2 \in \mathbb{R}_{>0}$ e $n_1 \in \mathbb{N}$ tali che

$$0 \leq c_1\, g(n) \leq f(n) \leq c_2\, g(n), \qquad \forall n \geq n_1.$$

Come si usano le definizioni

- Per dimostrare una di queste affermazioni **si determinano** le costanti (c, oppure c_1 e c_2) e la soglia n_1
- Per funzioni di due parametri, come $f(n, m)$, le definizioni sono le stesse, con le disuguaglianze richieste per ogni $n \geq n_1$ e ogni $m \geq m_1$
- I due fatti seguenti sono dimostrati nel capitolo «Notazione asintotica»

Osservazione 1: Θ equivale a O e Ω

Si ha $f(n) = \Theta\big(g(n)\big)$ *se e solo se* $f(n) = O\big(g(n)\big)$ *e* $f(n) = \Omega\big(g(n)\big)$.

Proposizione 1: criterio del limite

Sia $g(n) > 0$ *per ogni* $n \geq 1$. *Se*

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = L \in \mathbb{R}_{>0},$$

allora $f(n) = \Theta\big(g(n)\big)$.

Riepilogo

- O è un **limite superiore**, Ω un **limite inferiore**, Θ entrambi, per tutti gli n sufficientemente grandi
- Per dimostrarli si determinano le costanti e la soglia; con due parametri, una soglia per ciascuno
- Θ vale se e solo se valgono sia O sia Ω
- Un limite finito e positivo di $f(n)/g(n)$ dà $f(n) = \Theta\big(g(n)\big)$

Istanza di input costituita da un numero naturale

Introduzione: un numero naturale

- Partiamo dall'input più semplice: **un solo numero naturale**
- Dimostriamo che la sua dimensione dell'input cresce come $\log_2 n$
- È il primo esempio di una dimostrazione con le costanti c_1, c_2 e la soglia n_1

Dimensione dell'input di un numero naturale

- Se l'input è costituito da un solo numero naturale $n \geq 1$, la dimensione dell'input è

$$|I| = \lfloor \log_2 n \rfloor + 1,$$

che è il numero minimo di bit necessari per rappresentare n in binario

Osservazione 2: dimensione dell'input di un numero naturale

Si ha

$$\lfloor \log_2 n \rfloor + 1 = \Theta(\log_2 n).$$

Dimostrazione (1/2)

Per dimostrare il risultato, determiniamo $c_1, c_2 \in \mathbb{R}_{>0}$ e $n_1 \in \mathbb{N}$ tali che

$$0 \leq c_1 \log_2 n \leq \lfloor \log_2 n \rfloor + 1 \leq c_2 \log_2 n, \qquad \forall n \geq n_1.$$

Poiché $x < \lfloor x \rfloor + 1$ per ogni numero reale x, si ha

$$\log_2 n \leq \lfloor \log_2 n \rfloor + 1, \qquad \forall n \geq 1.$$

Dimostrazione (2/2)

Inoltre, poiché $\lfloor x \rfloor \leq x$ per ogni numero reale x, e $\log_2 n \geq 1$ per ogni $n \geq 2$, si ha

$$\lfloor \log_2 n \rfloor + 1 \leq \log_2 n + 1 \leq 2 \log_2 n, \qquad \forall n \geq 2.$$

Quindi, scegliendo $c_1 = 1$, $c_2 = 2$ e $n_1 = 2$, otteniamo

$$0 \leq \log_2 n \leq \lfloor \log_2 n \rfloor + 1 \leq 2 \log_2 n, \qquad \forall n \geq 2.$$

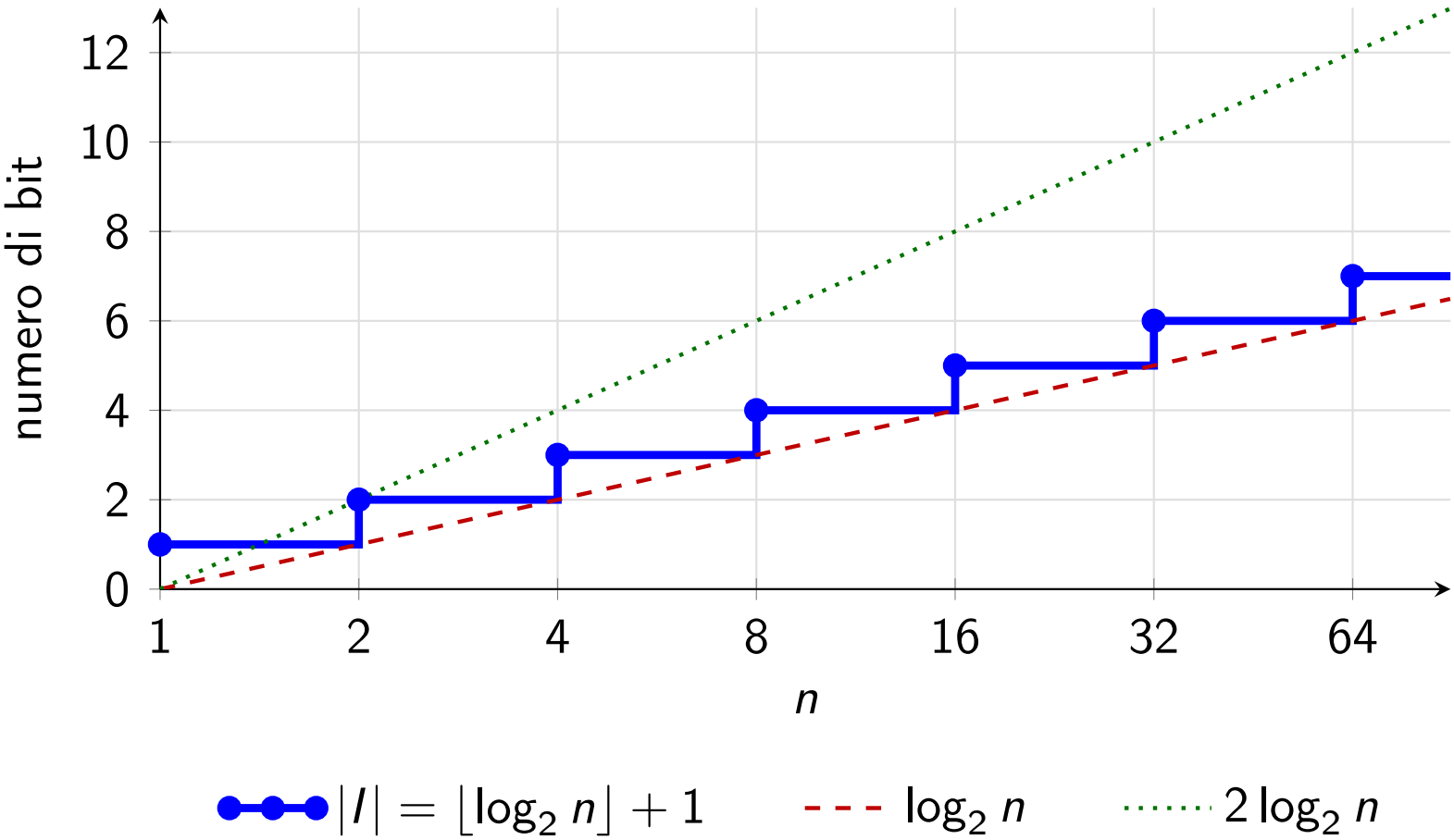
□

Un numero naturale: l'ordine di crescita

Se l'input di un algoritmo è costituito da
un solo numero naturale $n \geq 1$,
la dimensione dell'input soddisfa:

$$|I| = \Theta(\log_2 n).$$

La dimensione dell'input di un numero naturale



Come si legge il grafico

- Il grafico mostra la dimensione dell'input $|I| = \lfloor \log_2 n \rfloor + 1$ di un numero naturale n, con l'asse orizzontale in **scala logaritmica**
- Aumenta di **un bit a ogni potenza di** 2
- Per $n \geq 2$ è compresa fra $\log_2 n$ e $2 \log_2 n$, come afferma l'osservazione appena dimostrata

Esempio

- Per $n = 37$ si ha $|I| = 6$ (primo esempio del capitolo) e $\log_2 37 \approx 5.21$
- Le disuguaglianze della dimostrazione diventano

$$\log_2 37 \approx 5.21 \ \leq \ |I| = 6 \ \leq \ 2 \log_2 37 \approx 10.42.$$

- Per $n = 1$ si ha $|I| = 1$ ma $\log_2 1 = 0$: per questo la soglia della dimostrazione è $n_1 = 2$

Riepilogo

- Un numero naturale $n \geq 1$: $|I| = \lfloor \log_2 n \rfloor + 1$
- $|I| = \Theta(\log_2 n)$, con $c_1 = 1$, $c_2 = 2$ e $n_1 = 2$:

$$\log_2 n \leq \lfloor \log_2 n \rfloor + 1 \leq 2 \log_2 n, \qquad \forall n \geq 2$$

- La soglia $n_1 = 2$ serve perché $\log_2 1 = 0$

Istanza di input costituita da due numeri naturali

Introduzione: due numeri naturali

- Quando l'input è costituito da due numeri, vanno scritti i bit di **entrambi**
- Dimostriamo che la dimensione dell'input cresce come $\log_2 n + \log_2 m$
- e anche come il logaritmo del **maggiore** dei due numeri

Dimensione dell'input di due numeri naturali

- Se l'input è costituito da due numeri naturali $n, m \geq 1$, la dimensione dell'input è

$$|I| = \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big),$$

che è il numero minimo di bit necessari per rappresentare in binario sia n sia m

Osservazione 3: dimensione dell'input di due numeri naturali

Si ha

$$\left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) = \Theta(\log_2 n + \log_2 m).$$

Dimostrazione (1/4)

Per dimostrare il risultato, determiniamo $c_1, c_2 \in \mathbb{R}_{>0}$ e $n_1, m_1 \in \mathbb{N}$ tali che

$$\begin{aligned} 0 \leq c_1(\log_2 n + \log_2 m) &\leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \\ &\leq c_2(\log_2 n + \log_2 m) \end{aligned}$$

per ogni $n \geq n_1$ e $m \geq m_1$.

Dimostrazione (2/4)

Abbiamo

$$\log_2 n \leq \lfloor \log_2 n \rfloor + 1, \qquad \log_2 m \leq \lfloor \log_2 m \rfloor + 1, \qquad \forall n, m \geq 1.$$

Quindi

$$\log_2 n + \log_2 m \leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big), \qquad \forall n, m \geq 1.$$

Dimostrazione (3/4)

Inoltre

$$\lfloor \log_2 n \rfloor + 1 \leq \log_2 n + 1, \qquad \lfloor \log_2 m \rfloor + 1 \leq \log_2 m + 1.$$

Dunque

$$\big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \leq (\log_2 n + 1) + (\log_2 m + 1).$$

Se $n, m \geq 2$, allora $\log_2 n \geq 1$ e $\log_2 m \geq 1$, quindi

$$\log_2 n + 1 \leq 2 \log_2 n, \qquad \log_2 m + 1 \leq 2 \log_2 m.$$

Dimostrazione (4/4)

Pertanto

$$\big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \leq 2(\log_2 n + \log_2 m), \qquad \forall n, m \geq 2.$$

Quindi, scegliendo $c_1 = 1$, $c_2 = 2$, $n_1 = 2$ e $m_1 = 2$, otteniamo

$$\begin{aligned} 0 \leq \log_2 n + \log_2 m &\leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \\ &\leq 2(\log_2 n + \log_2 m), \qquad \forall n, m \geq 2. \end{aligned}$$

□

Due numeri naturali: l'ordine di crescita

Se l'input di un algoritmo è costituito da
due numeri naturali $n, m \geq 1$,
la dimensione dell'input soddisfa:

$$|I| = \Theta(\underbrace{\log_2 n + \log_2 m}_{\log_2(nm)}).$$

Il maggiore dei due numeri

La somma dei due logaritmi può essere sostituita
dal logaritmo del **maggiore** dei due numeri:

il numero maggiore richiede almeno metà dei bit dell'istanza.

Osservazione 4: dimensione dell'input di due numeri naturali e loro massimo

Si ha

$$\left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) = \Theta(\log_2 M),$$

dove

$$M = \max\{n, m\}.$$

Dimostrazione (1/4)

Per dimostrare il risultato, determiniamo $c_1, c_2 \in \mathbb{R}_{>0}$ e $n_1, m_1 \in \mathbb{N}$ tali che

$$\begin{aligned} 0 \leq c_1 \log_2 M &\leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \\ &\leq c_2 \log_2 M, \qquad \forall n \geq n_1,\ m \geq m_1. \end{aligned}$$

Poiché uno dei due numeri è uguale a M, si ha

$$\lfloor \log_2 M \rfloor + 1 \leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big).$$

Dimostrazione (2/4)

Inoltre,

$$\log_2 M \leq \lfloor \log_2 M \rfloor + 1, \qquad \forall M \geq 1.$$

Quindi,

$$\log_2 M \leq \big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big), \qquad \forall n, m \geq 1.$$

Dimostrazione (3/4)

D'altra parte, poiché $n \leq M$ e $m \leq M$, si ha

$$\lfloor \log_2 n \rfloor + 1 \leq \lfloor \log_2 M \rfloor + 1, \qquad \lfloor \log_2 m \rfloor + 1 \leq \lfloor \log_2 M \rfloor + 1.$$

Dunque

$$\big(\lfloor \log_2 n \rfloor + 1\big) + \big(\lfloor \log_2 m \rfloor + 1\big) \leq 2\big(\lfloor \log_2 M \rfloor + 1\big).$$

Se $M \geq 2$, allora $\log_2 M \geq 1$, quindi

$$\lfloor \log_2 M \rfloor + 1 \leq \log_2 M + 1 \leq 2 \log_2 M.$$

Dimostrazione (4/4)

Pertanto,

$$\left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) \leq 4 \log_2 M, \qquad \forall M \geq 2.$$

Quindi, scegliendo $c_1 = 1$, $c_2 = 4$, $n_1 = 2$ e $m_1 = 2$ (che danno $M \geq 2$), otteniamo

$$\begin{aligned} 0 \leq \log_2 M &\leq \left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right) \\ &\leq 4 \log_2 M, \qquad \forall n, m \geq 2. \end{aligned}$$

□

Due numeri naturali e il loro massimo

Se l'input di un algoritmo è costituito da
due numeri naturali $n, m \geq 1$,
la dimensione dell'input soddisfa:

$$|I| = \Theta\big(\log_2 M\big),$$

dove

$$M = \max\{n, m\}.$$

Esempio

- Per $n = 37$ e $m = 200$ si ha $|I| = 14$ (primo esempio del capitolo), $M = 200$, e

$$\log_2 37 + \log_2 200 \approx 12.85, \qquad \log_2 200 \approx 7.64.$$

- Le disuguaglianze delle dimostrazioni delle due osservazioni diventano

$$12.85 \leq 14 \leq 2 \cdot 12.85 = 25.71$$

e

$$7.64 \leq 14 \leq 4 \cdot 7.64 = 30.58,$$

dove tutti i valori sono arrotondati a due cifre decimali

Riepilogo

- Due numeri naturali $n, m \geq 1$:
  $|I| = \left(\lfloor \log_2 n \rfloor + 1\right) + \left(\lfloor \log_2 m \rfloor + 1\right)$
- $|I| = \Theta(\log_2 n + \log_2 m)$, con $c_1 = 1$, $c_2 = 2$, $n_1 = m_1 = 2$
- $|I| = \Theta(\log_2 M)$ con $M = \max\{n, m\}$, con $c_1 = 1$, $c_2 = 4$, $n_1 = m_1 = 2$: il numero maggiore richiede almeno metà dei bit

Istanza di input costituita da un array di numeri naturali

Introduzione: un array di numeri naturali

- La maggior parte degli algoritmi del corso riceve un **array**
- La sua dimensione dell'input dipende sia dal numero n di componenti sia dai loro valori
- Dimostriamo un limite superiore e un limite inferiore, vediamo quando coincidono, e guardiamo due varianti dell'input array

Dimensione dell'input di un array

- Se l'istanza di input è costituita da un array $a[1..n]$ di n numeri naturali, con $a[i] \geq 1$ per ogni $i \in \{1, 2, \ldots, n\}$, la dimensione dell'input è

$$|I| = \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right),$$

che è il numero minimo di bit necessari per rappresentare in binario tutte le componenti dell'array

Osservazione 5: limite superiore sulla dimensione dell'input di un array

Dato un array $a[1..n]$ *di* n *numeri naturali, con* $a[i] \geq 1$ *per ogni* $i \in \{1, 2, \ldots, n\}$*, si ha*

$$\sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) = O\big(n(1 + \log_2 M)\big),$$

dove

$$M = \max\big\{a[1], a[2], \ldots, a[n]\big\}.$$

Dimostrazione (1/2)

Per dimostrare il risultato, determiniamo $c \in \mathbb{R}_{>0}$ e $n_1 \in \mathbb{N}$ tali che

$$0 \leq \sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big) \leq c\, n \big(1 + \log_2 M \big), \qquad \forall n \geq n_1.$$

Poiché $a[i] \leq M$ per ogni $i \in \{1, 2, \ldots, n\}$, si ha

$$\lfloor \log_2 a[i] \rfloor + 1 \leq \lfloor \log_2 M \rfloor + 1, \qquad \forall i \in \{1, 2, \ldots, n\}.$$

Quindi,

$$\sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big) \leq \sum_{i=1}^{n} \big(\lfloor \log_2 M \rfloor + 1 \big) = n \big(\lfloor \log_2 M \rfloor + 1 \big).$$

Dimostrazione (2/2)

Inoltre, si ha

$$\lfloor \log_2 M \rfloor + 1 \leq \log_2 M + 1.$$

Dunque,

$$\sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) \leq n\left(1 + \log_2 M\right), \qquad \forall n \geq 1.$$

Quindi, scegliendo $c = 1$ e $n_1 = 1$, otteniamo

$$0 \leq \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) \leq n\left(1 + \log_2 M\right), \qquad \forall n \geq 1.$$

□

Un array: il limite superiore

Se l'input di un algoritmo è costituito da un array $a[1..n]$ di n numeri naturali, con $a[i] \geq 1$ per ogni $i \in \{1, 2, \ldots, n\}$, allora la dimensione dell'input soddisfa

$$|I| = O\big(n(1 + \log_2 M)\big),$$

dove

$$M = \max\big\{a[1], a[2], \ldots, a[n]\big\}.$$

Il limite superiore è stretto?

- Il limite superiore $n(1 + \log_2 M)$ conta ogni componente **come se fosse grande quanto il massimo**
- Quando solo poche componenti sono grandi, è una forte sovrastima
- Il limite non può essere sostituito da Θ: lo mostriamo con un controesempio

Esempio: il limite superiore non è stretto (1/4)

Dato un array $a[1..n]$ di n numeri naturali, con $a[i] \geq 1$ per ogni $i \in \{1, 2, \ldots, n\}$, per dimostrare che, in generale,

$$\sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) \neq \Theta\big(n(1 + \log_2 M)\big),$$

dove

$$M = \max\{a[1], a[2], \ldots, a[n]\},$$

è sufficiente fornire un **controesempio**.

- Consideriamo la famiglia di array definita da

$$a[1] = M, \qquad a[2] = a[3] = \cdots = a[n] = 1,$$

con $M \geq 2$.

Esempio: il limite superiore non è stretto (2/4)

- Per questi array, la dimensione dell'input è

$$|I| = \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right)$$

$$= (\lfloor \log_2 M \rfloor + 1) + (n-1)(\lfloor \log_2 1 \rfloor + 1).$$

- Poiché $\lfloor \log_2 1 \rfloor + 1 = 1$, otteniamo

$$|I| = (\lfloor \log_2 M \rfloor + 1) + (n-1) = n + \lfloor \log_2 M \rfloor.$$

Esempio: il limite superiore non è stretto (3/4)

- Scegliamo ora $M = 2^n$, in modo che $\log_2 M = n$
- Allora

$$|I| = n + n = 2n,$$

mentre

$$n(1 + \log_2 M) = n(1 + n) = n^2 + n.$$

Esempio: il limite superiore non è stretto (4/4)

- Se avessimo $2n = \Theta(n^2 + n)$, esisterebbero $c_1 \in \mathbb{R}_{>0}$ e $n_1 \in \mathbb{N}$ tali che

$$c_1(n^2 + n) \leq 2n, \qquad \forall n \geq n_1,$$

cioè

$$c_1(n + 1) \leq 2, \qquad \forall n \geq n_1,$$

il che è falso per n abbastanza grande

Quindi, per questa famiglia di array, $|I| \neq \Theta\big(n(1 + \log_2 M)\big)$.

Due casi particolari del limite superiore

- Se $M \geq 2$, allora $\log_2 M \geq 1$, e quindi $1 + \log_2 M \leq 2 \log_2 M$. Dunque

$$n(1 + \log_2 M) \leq 2n \log_2 M,$$

  e la dimensione dell'input soddisfa

$$|I| = O\big(n \log_2 M\big).$$

- Se $M = 1$, allora tutte le componenti dell'array sono uguali a 1, e quindi

$$|I| = \sum_{i=1}^{n} (\lfloor \log_2 1 \rfloor + 1) = n.$$

Verso un limite inferiore

Ogni componente dell'array richiede **almeno un bit**.

Questa semplice osservazione dà un limite inferiore
che vale per ogni array.

Osservazione 6: limite inferiore sulla dimensione dell'input di un array

Dato un array $a[1..n]$ *di* n *numeri naturali, con* $a[i] \geq 1$ *per ogni* $i \in \{1, 2, \ldots, n\}$*, si ha*

$$\sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) = \Omega(n).$$

Dimostrazione (1/2)

Per dimostrare il risultato, determiniamo $c \in \mathbb{R}_{>0}$ e $n_1 \in \mathbb{N}$ tali che

$$0 \leq c\, n \leq \sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big), \qquad \forall n \geq n_1.$$

Poiché $a[i] \geq 1$ per ogni $i \in \{1, 2, \ldots, n\}$, si ha

$$\lfloor \log_2 a[i] \rfloor + 1 \geq \lfloor \log_2 1 \rfloor + 1 = 1, \qquad \forall i \in \{1, 2, \ldots, n\}.$$

Quindi,

$$\sum_{i=1}^{n} \big(\lfloor \log_2 a[i] \rfloor + 1 \big) \geq \sum_{i=1}^{n} 1 = n.$$

Dimostrazione (2/2)

Quindi, scegliendo $c = 1$ e $n_1 = 1$, otteniamo

$$0 \leq n \leq \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right), \qquad \forall n \geq 1.$$

□

Un array: il limite inferiore

Se l'input di un algoritmo è costituito da un array $a[1..n]$ di n numeri naturali, con $a[i] \geq 1$ per ogni $i \in \{1, 2, \ldots, n\}$, allora la dimensione dell'input soddisfa

$$|I| = \Omega(n).$$

Esempio

- Per l'array $a = [6, 1, 3, 8]$ del primo esempio del capitolo si ha

$$n = 4, \qquad |I| = 10, \qquad M = 8$$

- Il limite inferiore e il limite superiore delle due osservazioni danno

$$n = 4 \ \leq \ |I| = 10 \ \leq \ n(1 + \log_2 M) = 4\,(1 + 3) = 16.$$

Quando i due limiti coincidono

In molte applicazioni le componenti dell'array sono memorizzate con un **numero fisso di bit**, indipendente da n.

In questo caso i due limiti coincidono.

Corollario 1: componenti di dimensione costante

Dato un array $a[1..n]$ *di* n *numeri naturali, con* $a[i] \geq 1$ *per ogni* $i \in \{1, 2, \ldots, n\}$*, se tutte le componenti dell'array possono essere rappresentate usando al più un numero costante fissato di bit, allora*

$$|I| = \Theta(n).$$

Dimostrazione

Per ipotesi esiste una costante $c \in \mathbb{N}$ tale che

$$\lfloor \log_2 a[i] \rfloor + 1 \leq c, \qquad \forall i \in \{1, 2, \ldots, n\}.$$

Quindi,

$$|I| = \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right) \leq \sum_{i=1}^{n} c = cn, \qquad \forall n \geq 1,$$

e dunque $|I| = O(n)$.

Per il limite inferiore sulla dimensione dell'input di un array si ha anche $|I| = \Omega(n)$, e quindi, poiché Θ equivale a O e Ω,

$$|I| = \Theta(n).$$



Due varianti dell'input array

- I limiti visti si estendono a due varianti dell'input array:
  - l'input contiene anche il **numero** n di componenti
  - le componenti dell'array possono essere 0

- In entrambi i casi valgono gli stessi ordini di crescita

Il numero n e l'array (1/3)

- Se l'input è costituito da un numero naturale $n \geq 1$ e da un array $a[1..n]$ di n numeri naturali, con $a[i] \geq 1$ per ogni $i \in \{1, 2, \ldots, n\}$, la dimensione dell'input è

$$|I| = \left(\lfloor \log_2 n \rfloor + 1\right) + \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1\right).$$

- Sia

$$M = \max\{n, a[1], a[2], \ldots, a[n]\}.$$

Il numero n e l'array (2/3)

Allora si ha

$$\lfloor \log_2 n \rfloor + 1 \leq 1 + \log_2 M,$$

e, per ogni $i \in \{1, 2, \ldots, n\}$,

$$\lfloor \log_2 a[i] \rfloor + 1 \leq 1 + \log_2 M.$$

Quindi,

$$|I| \leq (1 + \log_2 M) + n(1 + \log_2 M) = (n + 1)(1 + \log_2 M).$$

Dunque

$$|I| = O\big((n + 1)(1 + \log_2 M)\big).$$

Il numero n e l'array (3/3)

- Poiché $n + 1 \leq 2n$ per ogni $n \geq 1$, otteniamo anche

$$|I| = O\big(n(1 + \log_2 M)\big).$$

- Inoltre, se $M \geq 2$, allora $1 + \log_2 M \leq 2 \log_2 M$, e quindi

$$|I| = O\big(n \log_2 M\big).$$

- Il limite inferiore $|I| = \Omega(n)$ continua a valere, poiché i soli bit dell'array sono almeno n

Componenti uguali a 0 (1/3)

- Se le componenti dell'array possono essere 0, la dimensione dell'input è

$$|I| = \sum_{i=1}^{n} \left(\lfloor \log_2(\max\{1, a[i]\}) \rfloor + 1 \right),$$

così che ogni componente uguale a 0 contribuisce esattamente con 1 bit

Componenti uguali a 0 (2/3)

- In questo caso, se poniamo

$$M = \max\{1, a[1], a[2], \ldots, a[n]\},$$

le formule precedenti diventano

$$|I| = O\big(n(1 + \log_2 M)\big),$$

e, se $M \geq 2$,

$$|I| = O(n \log_2 M).$$

Componenti uguali a 0 (3/3)

- Inoltre, continua a valere

$$|I| = \Omega(n),$$

poiché ogni componente contribuisce con almeno 1 bit

- In particolare, se tutte le componenti dell'array possono essere rappresentate usando al più un numero costante fissato di bit, allora

$$|I| = \Theta(n).$$

Riepilogo (1/2)

Array $a[1..n]$ con componenti $a[i] \geq 1$:

- $|I| = \sum_{i=1}^{n} \left(\lfloor \log_2 a[i] \rfloor + 1 \right)$
- Limite superiore: $|I| = O\big(n(1 + \log_2 M)\big)$, con M la componente massima, e $|I| = O(n \log_2 M)$ se $M \geq 2$
- Il limite superiore in generale **non è stretto**: $a[1] = 2^n$ e tutte le altre componenti uguali a 1 danno $|I| = 2n$

Limite inferiore:

- $|I| = \Omega(n)$, perché ogni componente richiede almeno un bit

Riepilogo (2/2)

Componenti di dimensione costante:

- Se ogni componente ha al più un numero costante di bit, $|I| = \Theta(n)$

Due varianti:

- Con il numero n nell'input, e $M = \max\{n, a[1], \ldots, a[n]\}$, valgono gli stessi limiti $O\big(n(1 + \log_2 M)\big)$ e $\Omega(n)$
- Con componenti uguali a 0, ciascuna conta 1 bit, e valgono gli stessi limiti con $M = \max\{1, a[1], \ldots, a[n]\}$

Dimensione dell'input e numero di passi

Introduzione: Dimensione dell'input e numero di passi

- Possiamo ora spiegare **perché la dimensione dell'input si misura in bit**
- Definiamo il tempo di esecuzione nel caso peggiore, e gli algoritmi polinomiali ed esponenziali
- Vediamo che lo stesso numero di passi può essere polinomiale o esponenziale, a seconda del ruolo del parametro n

Contare i passi

- Il tempo richiesto da un algoritmo si stima contando i suoi **passi**:
  - nello pseudocodice, un passo è l'esecuzione di una riga
  - la condizione di un ciclo **while** conta come un passo ogni volta che viene valutata

- Il numero di passi viene
  - calcolato prima come funzione $S(n)$ di un parametro naturale n dell'input
  - poi espresso come funzione della dimensione dell'input $|I|$

Definizione 2: tempo di esecuzione nel caso peggiore

Il ***tempo di esecuzione nel caso peggiore*** $T(|I|)$ *di un algoritmo è il massimo numero di passi eseguiti dall'algoritmo su un qualsiasi input* I *di una data dimensione* $|I|$.

Definizione 3: algoritmi polinomiali ed esponenziali

Un algoritmo è ***polinomiale*** *se il suo tempo di esecuzione nel caso peggiore soddisfa* $T(|I|) = O(|I|^k)$ *per una qualche costante* $k \in \mathbb{N}$.

Un algoritmo è ***esponenziale*** *se il suo tempo di esecuzione nel caso peggiore soddisfa* $T(|I|) = 2^{\Theta(|I|)}$, *cioè se esistono* $c_1, c_2 \in \mathbb{R}_{>0}$ *e* $k_1 \in \mathbb{N}$ *tali che*

$$2^{c_1|I|} \leq T(|I|) \leq 2^{c_2|I|}, \qquad \forall |I| \geq k_1.$$

Due ruoli del parametro n

- Il parametro n può avere due ruoli molto diversi:
  - il **valore** di un numero dato in input
  - il **numero di componenti** di un array dato in input

- L'osservazione seguente mostra come n è legato a $|I|$ nei due casi

Osservazione 7: valore di un numero e numero di componenti di un array

1. *Se l'input è costituito da un solo numero naturale* $n \geq 1$, *allora*

$$2^{|I|-1} \leq n \leq 2^{|I|} - 1 :$$

il valore n *è esponenziale nella dimensione dell'input.*

2. *Se l'input è costituito da un array* $a[1..n]$ *di* n *numeri naturali (ed eventualmente dal numero* n*), allora*

$$n \leq |I| :$$

il numero di componenti n *è al più lineare nella dimensione dell'input.*

Dimostrazione

1. Poiché $|I| = \lfloor \log_2 n \rfloor + 1$, si ha $\lfloor \log_2 n \rfloor = |I| - 1$, cioè

$$|I| - 1 \leq \log_2 n < |I|.$$

   La funzione $x \mapsto 2^x$ è strettamente crescente, quindi $2^{|I|-1} \leq n < 2^{|I|}$. Poiché n è un intero, $n \leq 2^{|I|} - 1$.

2. Ogni componente dell'array contribuisce con almeno un bit alla dimensione dell'input (limite inferiore sulla dimensione dell'input di un array, e lo stesso vale per le componenti uguali a 0), quindi $|I| \geq n$.

□

Fattori costanti

Quando un numero di passi è limitato da esponenziali
con fattori costanti, i fattori costanti
possono essere **assorbiti nell'esponente**.

Osservazione 8: fattori costanti nell'esponente

Siano $a_1, a_2, b_1, b_2 \in \mathbb{R}_{>0}$. *Se una funzione* T *soddisfa*

$$a_1\, 2^{b_1 x} \leq T(x) \leq a_2\, 2^{b_2 x}$$

per ogni numero naturale x *sufficientemente grande, allora* $T(x) = 2^{\Theta(x)}$.

Dimostrazione (1/2)

Scriviamo i fattori costanti come potenze di 2:

$$a_1 = 2^{\log_2 a_1}, \qquad a_2 = 2^{\log_2 a_2}.$$

Allora

$$a_2\, 2^{b_2 x} = 2^{b_2 x + \log_2 a_2} \leq 2^{2 b_2 x} \quad \text{ogni volta che} \quad b_2 x \geq \log_2 a_2,$$

e

$$a_1\, 2^{b_1 x} = 2^{b_1 x + \log_2 a_1} \geq 2^{\frac{b_1}{2} x} \quad \text{ogni volta che} \quad \frac{b_1}{2}\, x \geq -\log_2 a_1.$$

Dimostrazione (2/2)

- Entrambe le condizioni valgono per ogni x sufficientemente grande
- Quindi, con $c_1 = \frac{b_1}{2}$ e $c_2 = 2b_2$, otteniamo

$$2^{c_1 x} \leq T(x) \leq 2^{c_2 x}$$

per ogni x sufficientemente grande, cioè $T(x) = 2^{\Theta(x)}$

□

Esempio: la somma delle componenti di un array (1/5)

- Consideriamo l'algoritmo `SumArray` della prossima slide, che calcola la somma delle componenti di un array
- Contiamo i suoi passi in funzione del numero n di componenti
- Poi li esprimiamo in funzione della dimensione dell'input $|I|$

Esempio: la somma delle componenti di un array (2/5)

Algorithm: SumArray

Input : Un array $a[1..n]$ di $n \geq 1$ numeri naturali
Output : $a[1] + a[2] + \cdots + a[n]$

```
1 function SumArray(a, n)
2     s := 0
3     i := 1
4     while i ≤ n do
5         s ← s + a[i]
6         i ← i + 1
7     end
8     return s
9 end
```

Esegui in Colab: Python · C++ · Java

Esempio: la somma delle componenti di un array (3/5)

Passo 1: numero di passi.

- Le righe 2, 3 e 8 sono eseguite 1 volta ciascuna
- La condizione alla riga 4 è valutata $n + 1$ volte (l'ultima volta è falsa)
- Le righe 5 e 6 sono eseguite n volte ciascuna

Quindi

$$S(n) = 3 + (n + 1) + 2n = 3n + 4,$$

per ogni array di n componenti, qualunque siano i suoi valori.

Esempio: la somma delle componenti di un array (4/5)

Passo 2: da n a $|I|$.

- Ogni input di dimensione $|I|$ ha $n \leq |I|$ componenti (numero di componenti di un array, nell'osservazione vista sopra), quindi richiede al più $3|I| + 4$ passi
- L'array di $|I|$ componenti tutte uguali a 1 ha dimensione dell'input $|I|$ e richiede esattamente $3|I| + 4$ passi

Dunque

$$T(|I|) = 3|I| + 4 = O(|I|).$$

Esempio: la somma delle componenti di un array (5/5)

Passo 3: conclusione.

- `SumArray` è **polinomiale** (lineare) nella dimensione dell'input
- Se invece lo stesso ciclo andasse da 1 al *valore* di un numero n dato in input, i $\Theta(n)$ passi sarebbero **esponenziali** nella dimensione dell'input
- perché n può arrivare a $2^{|I|} - 1$ (valore di un numero, nell'osservazione vista sopra)

Riepilogo

- Il tempo di esecuzione nel caso peggiore $T(|I|)$ è il massimo numero di passi sugli input di dimensione $|I|$
- **Polinomiale**: $T(|I|) = O(|I|^k)$; **esponenziale**: $T(|I|) = 2^{\Theta(|I|)}$, e i fattori costanti si assorbono nell'esponente
- Il **valore** di un numero è esponenziale in $|I|$; il **numero di componenti** di un array è al più $|I|$
- `SumArray` esegue $3n + 4$ passi ed è lineare nella dimensione dell'input

Riepilogo (1/2)

Punti chiave del capitolo

- La dimensione dell'input $|I|$ è il numero di bit necessari per scrivere in binario l'istanza di input
- Un numero naturale n: $|I| = \lfloor \log_2 n \rfloor + 1 = \Theta(\log_2 n)$
- Due numeri naturali n, m:
  $|I| = \Theta(\log_2 n + \log_2 m) = \Theta(\log_2 \max\{n, m\})$

Riepilogo (2/2)

- Un array di n numeri naturali: $|I| = \Omega(n)$ e $|I| = O\big(n(1 + \log_2 M)\big)$, con M la componente massima; il limite superiore in generale non è stretto. Se le componenti hanno un numero costante di bit, $|I| = \Theta(n)$

- Lo stesso numero di passi $S(n)$ può essere polinomiale o esponenziale in $|I|$: il valore di un numero è esponenziale nella sua dimensione dell'input, mentre il numero di componenti di un array è al più la sua dimensione dell'input